

COMP90041

Programming and Software Development

Lecture 4 - Classes and Methods - I

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Melbourne in accordance with section 113P of the *Copyright Act 1968 (Act)*.

The material in this communication may be subject to copyright under the Act.

Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice



THE UNIVERSITY OF
MELBOURNE

Quick Recap

Quick Recap



```
public class Car {  
    public String modelName = "Toyota";  
    public String getModelName() {  
        return modelName;    }  
}  
public static void main() {  
    Car myCar = new Car();  
    String myCarName = myCar.getModelName();  
}
```

primitive variable

method

create an object / it is an instance of class Car

reference variable

Quick Recap



```
public class Car {  
    public String modelName = "Toyota";  
    public String getModelName() {  
        return modelName;  
    }  
    public static void main() {  
        myCarName = getModelName();  
    }  
}
```

?

Why does this fail?



THE UNIVERSITY OF
MELBOURNE

Lecture 4

Topics:

- **Classes and Methods**
- **References and class parameters**



Classes and Methods



Overview

- Access modifiers (e.g., public vs private)
- Accessor and mutator methods
- Overloading
- Static methods and static variables

Access Modifiers



- Control the **visibility** of class members (variables and methods).
- Essential for **encapsulation** and information hiding.
- Type of access modifiers:
 - Public: **No** restrictions; accessible everywhere
 - Private: Accessible only **within** its own class
 - Protected: applies only with **inheritance** (Will cover it later)
 - Default: (Will cover it later)

Accessors and Mutators



- Instance variables should be **private by default**.
- Need ways to access or modify them in a controlled way.
- Accessor methods ("**getters**") retrieve data.
- Mutator methods ("**setters**") modify data.

Accessor Methods ("Getters")



- They allow **obtaining the value** of an object's instance variables.
- The data can be **accessed but not changed**.
- The name of an accessor method typically starts with the word **get**.
- Can compute values before returning.

```
public class Car {  
    private String manufacturer;  
    private int yearBuilt;  
    // accessor methods  
    public String getManufacturer() {  
        return manufacturer;  
    }  
    public int getYearBuilt() {  
        return yearBuilt;  
    }  
}
```

Mutator Method ("Setters")



- Allow **controlled modification** of private variables.
- Usually start with **set**.
- Can **validate input** before updating variables.

```
public class Car {  
    private String manufacturer;  
    private int yearBuilt;  
    // mutator methods  
    public void setManufacturer(String manufacturer) {  
        this.manufacturer = manufacturer;  
    }  
    public void setYearBuilt(int yearBuilt) {  
        if (yearBuilt >= 0) {  
            this.yearBuilt = yearBuilt;  
        }  
    }  
}
```

Overloading



- **Overloading**: two or more methods in **same** class with **same** name.
- Each method must have a unique **signature**.
- A **signature** consists of the name of a method together with its parameter list.

```
// method takes one parameter of type Car  
public void setCar(Car otherCar) { ... }
```

```
// method takes two parameters  
public void setCar(String manufacturer, int yearBuilt) { ... }
```

```
// method takes two parameters of different types  
public void setCar(String manufacturer, String yearBuilt) { ... }
```

Overloading



- Different signatures must have **different** numbers and/or types of parameters.
- If we call the method in main as `myCar.setCar(yourCar)`, which of the following methods will be invoked?

```
public void setCar(Car otherCar) { ... }  
  
// method with the same signature but different return type  
public String setCar(Car otherCar) { ... }
```

- Methods with same name & parameters but **different return** types are **not** allowed.
- Although many programming languages, such as C++, allow you to overload operators (+, -, etc.), Java **does not permit** this.
- If exact signature **not found**, Java tries **automatic type conversion**, which can lead to ambiguous calls and result in a compilation error.

Static Methods



- Can be used without creating an object
- Still belongs to a class, and defined inside the class
- Declared using the **static** keyword

```
class MyClass {  
    public static returnType myMethod (parameters) {  
        // method body  
    }  
}
```

- Called using the **class name**:

```
returnedValue = MyClass.myMethod(arguments);
```


Static Methods



- ⚠ Static methods **cannot** call non-static variables and methods
- What is wrong with the following code?

```
class MyClass {  
    private void sayHello () {  
        System.out.println ("Hello, World!");  
    }  
    public static void main (String[] args) {  
        sayHello ();  
    }  
}
```

Static Methods



How to fix the previous example?

- **Option 1:** Make sayHello() **static**:

```
private static void sayHello () {  
    System.out.println ("Hello, World!");  
}
```

- **Option 2:** Create an **object** in main()

```
public static void main (String[] args) {  
    MyClass instance = new MyClass();  
    instance.sayHello ();  
    // or  
    sayHello();  
}
```

Main in any class



- The main() method can be in:
 - A separate class, or
 - A regular class
- When you have **multiple** classes, **each** class can have their **own** main method
- How does the system know where to start?
 - It **depends** on how the compiler is called
 - Example: `java MyClass` → Starts from `MyClass.java`'s `main()` method
- Why **more than one** `main()`?
 - For self-tests or diagnostics
 - Useful when each class is compiled/tested independently

What is the output of the command `java MyClass`?



```
class MyClass {  
    public static void main (String[] args) {  
        System.out.println ("Hello, World!");  
        Other.main(new String[0]);  
    }  
}  
class Other {  
    public static void main (String[] args) {  
        System.out.println ("Goodbye.");  
    }  
}
```

The Output will be:

```
Hello, world!  
Goodbye.
```

Static Variables



- A **static variable** belongs to the **class**, not any specific object
- Static vs Instance Variables:
 - A static variable → Only **one** copy **per class**, shared by all instances
 - Instance variable → **one** copy **per object**.
- All objects of the class can read and modify the same static variable
- Declared like an instance variable, but with **static**

```
private static int myStaticVariable;
```

Static Variables

- What is the output of the following?

```
class Test {  
    static int count = 0;  
  
    Test() {  
        count++;  
    }  
  
    public static void main (String[] args) {  
        new Test();  
        new Test();  
        System.out.println(count);  
    }  
}
```

The Output will be:

2

Static Variables



- ⚠ Static methods **can access** static variables, but **cannot** access **instance variables**
- What is the output of the following code?

```
class MyClass {  
    private int instanceVar = 10;  
    private static int staticVar = 20;  
  
    public static void main (String[] args) {  
        System.out.println(staticVar);  
        System.out.println(instanceVar);  
    }  
}
```

Static Variable Initialization



- Static variables can be declared and initialized at the same time

```
private static int myStaticVariable = 0;
```

- If not explicitly initialized, they get **default** values
 - boolean → false
 - Numeric types (int, double, etc.) → 0 of their type
 - Object/class types → null
- It is **good practice** to **initialize** static variables explicitly

Constants variables



- A constant variable a variable whose value **cannot change** after it's assigned.
- A constant is defined using the **final** keyword

```
public static final double PI = 3.14159;
```

- While it recommended to make your variables in general private, constant variables are safe to be **public**

Live Coding – Part 1

(getter & setter & static variable & static method)

Live Coding Instruction

Implement a Person



Write a Java class called *Person* that has three instance variables:

- a name (String)
- an age (int)
- a gender (String)

Additionally, implement a constructor that initializes these instance variables when called as follows:

```
Person person = new Person("Joe", 25, "male");
```

Make sure that a person's age is never < 0 . The default value for *age* should be set to 0, the name to "anonymous", and the gender to "unknown" if any invalid arguments are provided.

For each instance variable, make sure to provide the following accessor methods:

```
public String getName() {...}  
public int getAge() {...}  
public String getGender() {...}
```

Live Coding Instruction

Don't forget the *setter* methods:

```
public void setName(String name) {...}  
public void setAge(int age) {...}  
public void setGender(String gender) {...}
```

Add a *toString()* method that returns the value of the instance variables as follows:

```
Joe (25 years old), gender: male
```

Also, add an *equals()* method that takes another *Person* object as a parameter and returns *true* if all instance variable values are equal.

```
public boolean equals(Person anotherPerson) {}
```

The *Program.java* file contains the program entry (*i.e.*, the main method). You should not modify it. Instead, you need to modify your *Person.java* class in order to make the program run correctly.

Extension: Make gender case-insensitive, so that "Female" matches "female". You may want to use `String.toLowerCase(String s)`.



Live Coding Instruction



Define a class MyClass with a static String variable and a non-static String variable.

In the “public static void main()” of this class, create two instances of this class, change the values of these two variables in the first instance, and check these values in the second instance.



THE UNIVERSITY OF
MELBOURNE

Break

References and class parameters



Overview

- Variables and Memory
- Call-by-Value vs Call-by-Reference
- Using and misusing references

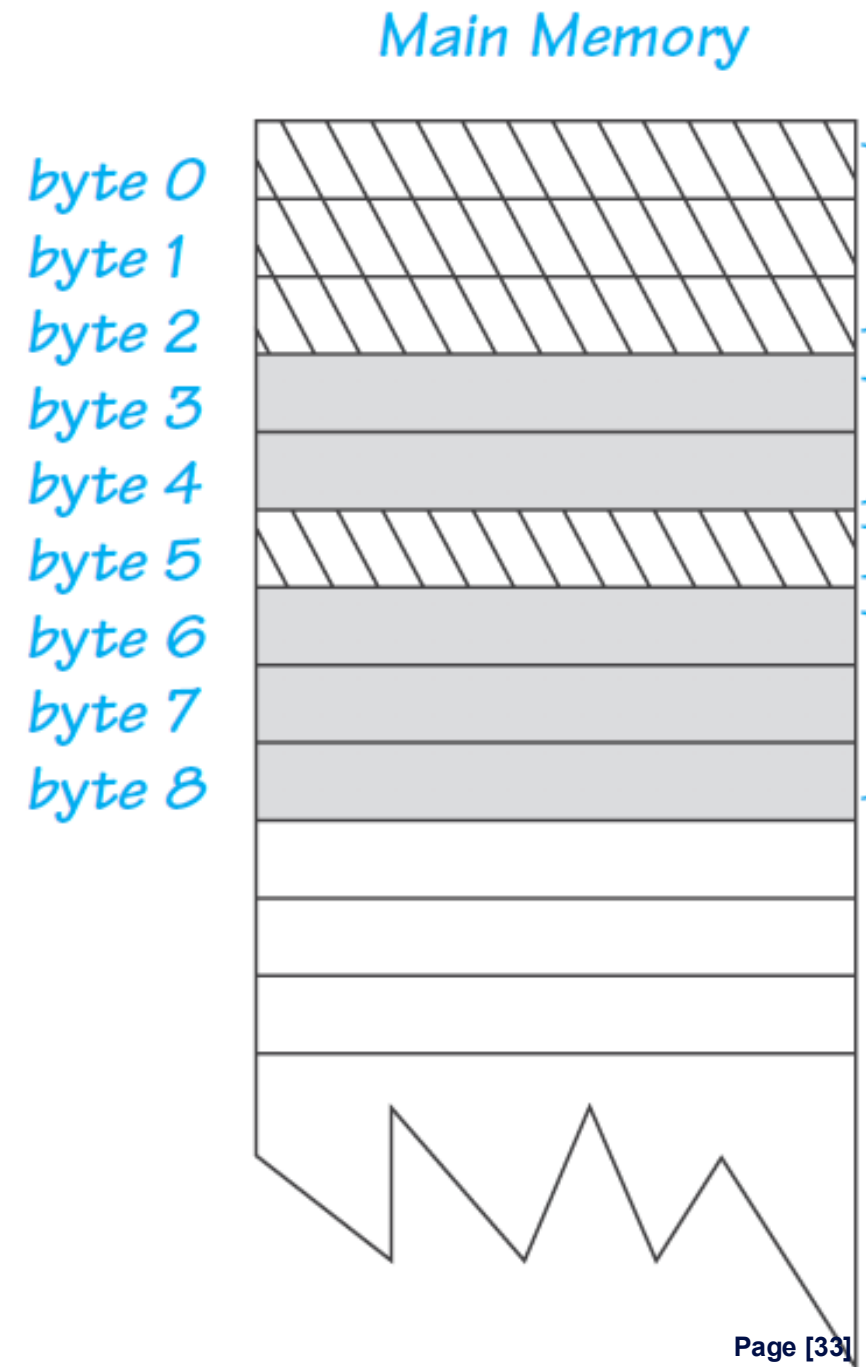
Storing Data



- Computers use two main types of memory:
 - Secondary Memory
 - Hard disk or SSD (Solid State Drive)
 - Used for long-term storage (files, saved data)
 - Main Memory (Primary Memory)
 - Typically, RAM (Random Access Memory)
 - Used while running a program
- Values stored in a **program's variables** are kept in **main memory**.

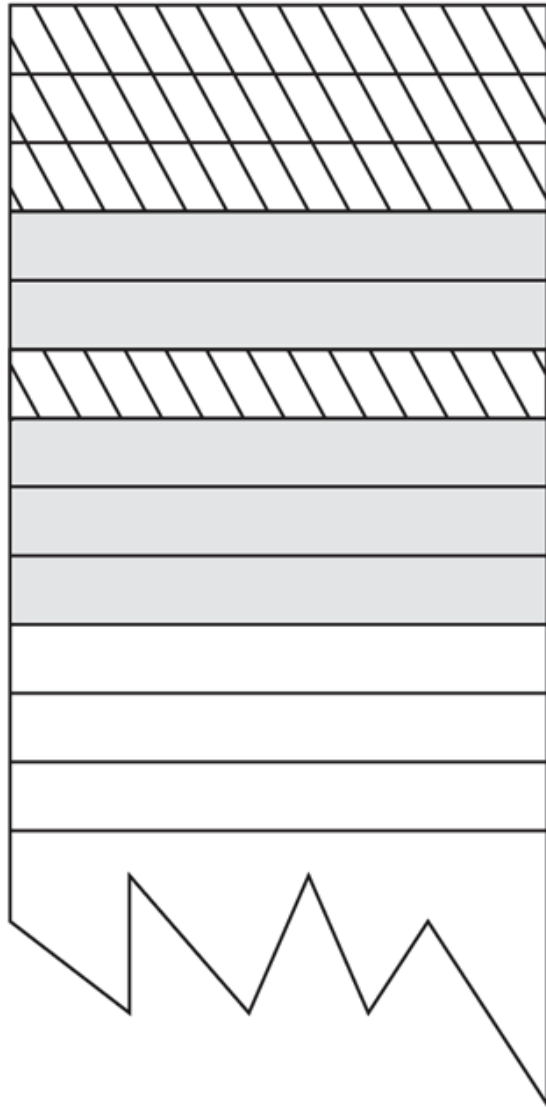
Variables and memory

- Main memory is a **long list** of numbered byte **locations**
 - Each byte = 8 bits → stores a number from 0 to 255
 - Each location has an address (e.g., 64-bit integer on modern systems)
- A **variable's value** is **stored** in one or (typically) more bytes (even for 1-bit boolean variables)
- The system uses the address of the first byte of the variable to:
 - Read the value
 - Write a new value



Main Memory

byte 0
byte 1
byte 2
byte 3
byte 4
byte 5
byte 6
byte 7
byte 8



variable1 (3-byte location with address 0)

variable2 (2-byte location with address 3)

variable3 (1-byte location with address 5)

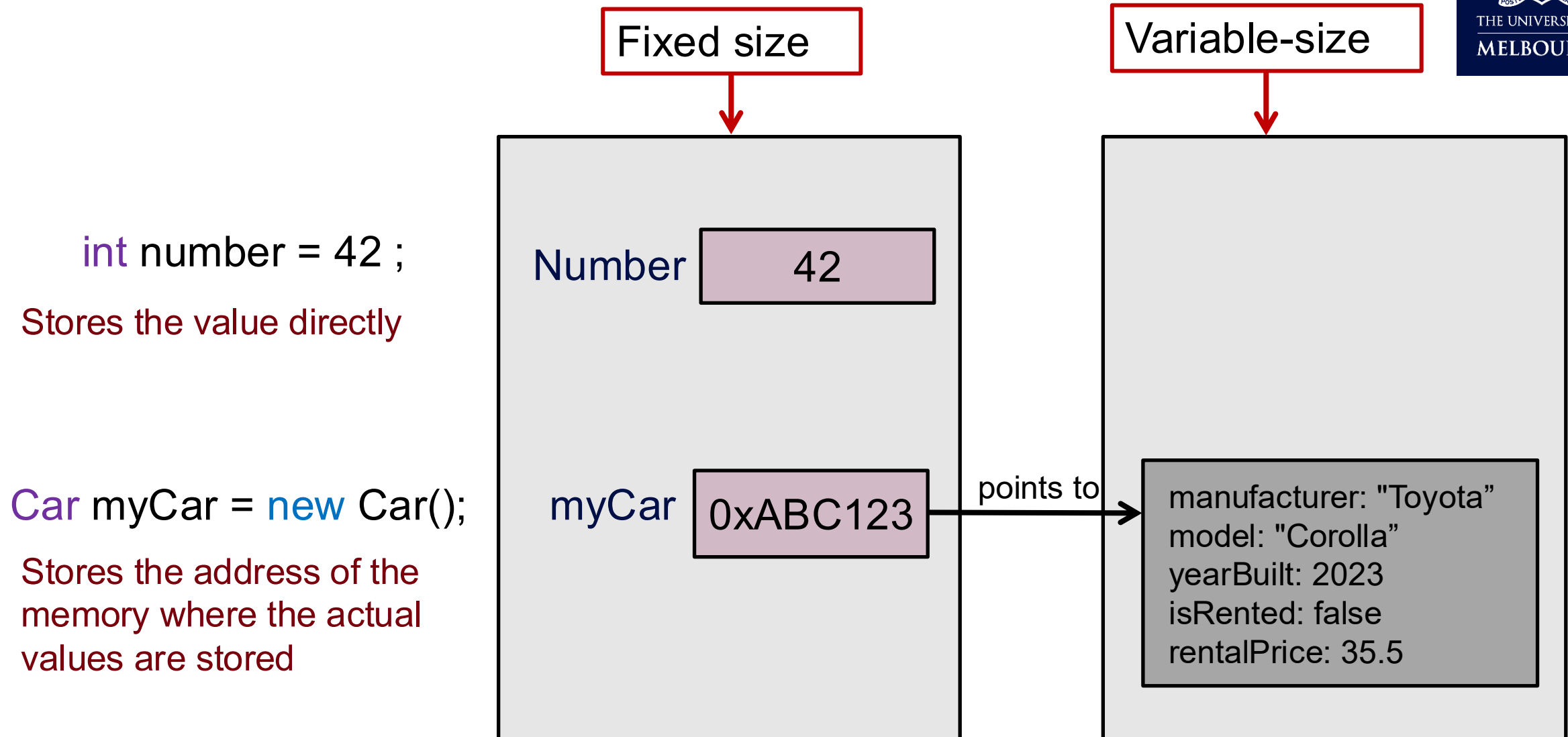
variable4 (3-byte location with address 6)

How Variables Are Stored in Memory



- Primitive Types (e.g., int, double)
 - The actual value is **stored directly** in the memory location
 - Always require a **fixed** amount of memory
- Reference Types (Classes/Objects)
 - The variable stores a **reference** (memory address), not the object itself
 - The object is stored elsewhere in memory
 - **Reference size** is **fixed**, but the object can be **variable-sized**

How Variables Are Stored in Memory



How Variables Are Stored in Memory

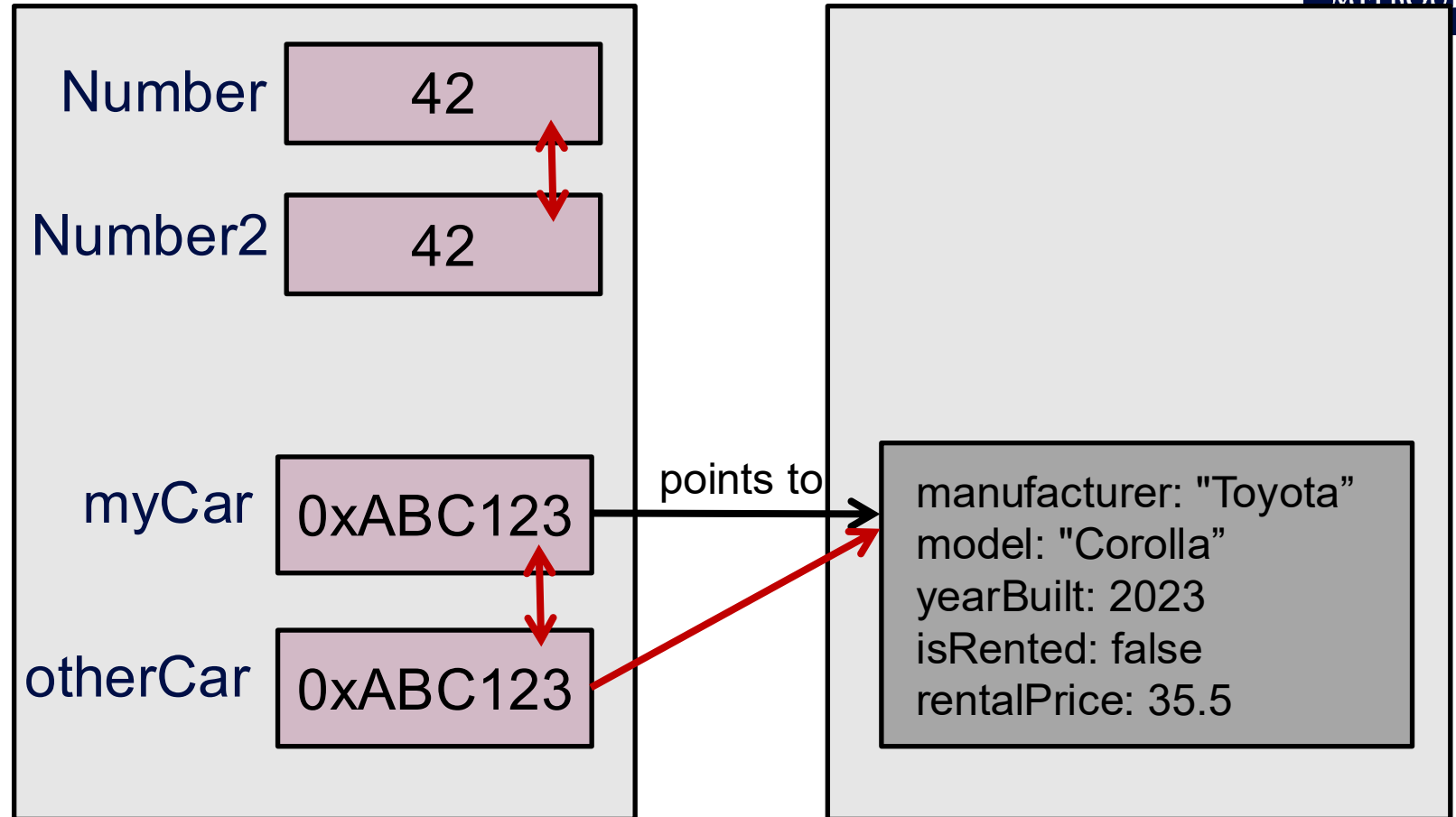
```
int number = 42 ;  
int number2 = number;
```

Copies the value (Actual value)

// Assume myCar is initialized
in the constructor

```
Car myCar = new Car();  
Car otherCar = myCar;
```

Copies the value
(address/reference)



Any change to the object named by one of these variables will produce a change to the object named by the other variable

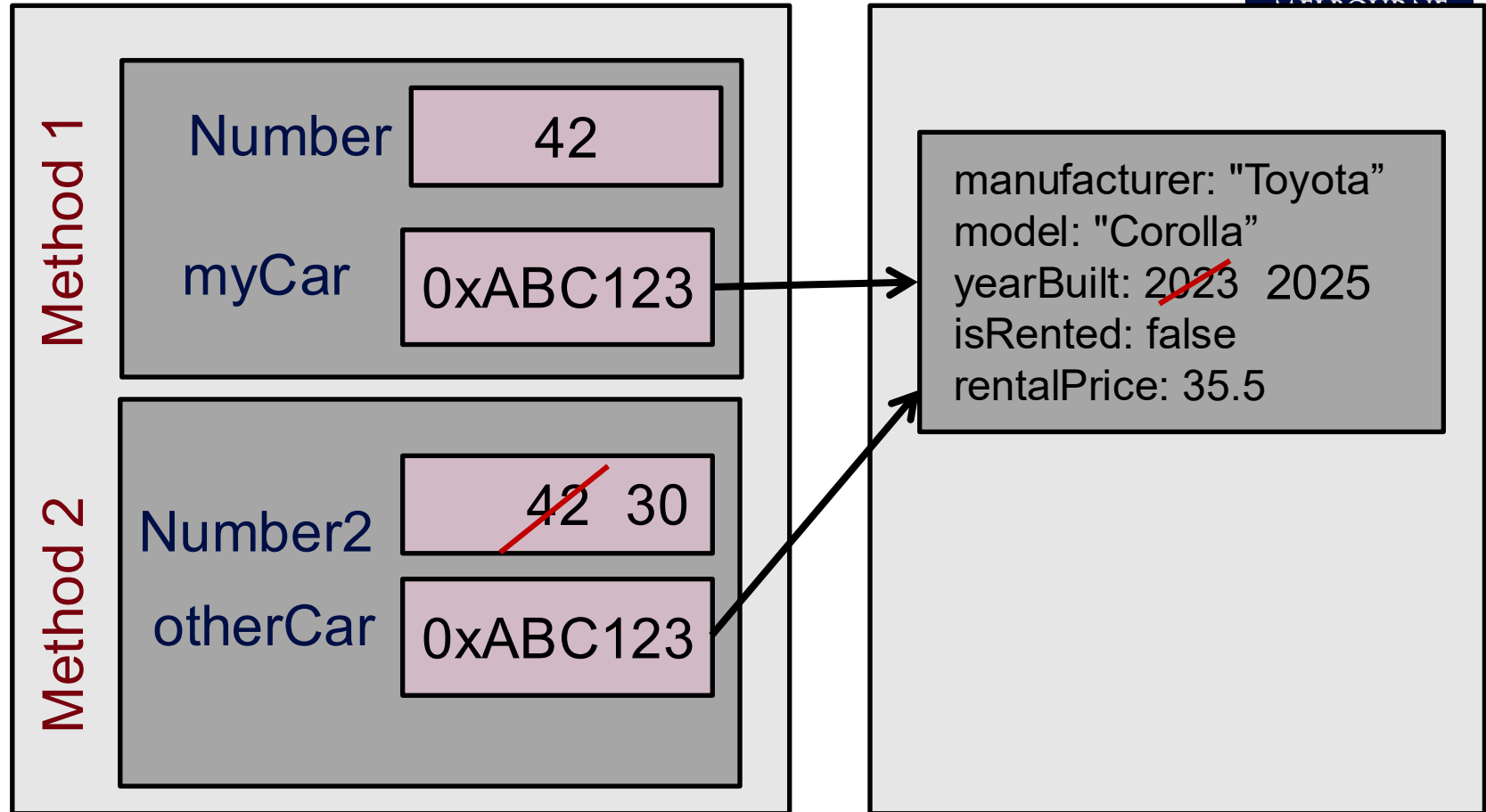
Call-by-Value vs Call-by-Reference



- There are two ways to pass arguments into a method:
 - Call-by-Value: A copy of the **value** is passed
 - Call-by-Reference: A **reference** to the original is passed

Call-by-Value vs Call-by-Reference

```
public void Method1 () {  
    int Number = 42 ;  
    // Assume myCar is initialized  
    // in the constructor  
    Car myCar = new Car();  
  
    Method2 (Number , myCar);  
}
```



`otherCar.yearBuilt = 2025;`

Call-by-Value vs Call-by-Reference



```
public class Car {  
    private float price;  
    public void changePrice (float x) {  
        x= 200;  
    }  
    public void changePrice (Car otherCar) {  
        otherCar.price = 200;  
    }  
    public static void main (String[] args) {  
        float p = 100;  
        changePrice(p);  
        System.out.println("Price = " + p);  
        Car c = new Car();  
        c.price = 100;  
        changePrice(c);  
        System.out.println("Car price = " + c.price);  
    }  
}
```

The Output will be:

```
Price = 100  
Car price = 200
```


Strings: A Special Case of Passing by Value



- Java is always pass-by-value
 - For objects (like String), Java passes a copy of the reference
 - For primitives, it passes the actual value

```
public static void modify (String s) {  
    s = s + " world";  
}  
  
public static void main (String[] args) {  
    String s = "hello";  
    modify(s);  
    System.out.println(s);  
}
```

The Output will be:

hello

- Strings are **immutable**
 - You **can't change** the original String through the reference
 - Any "**modification**" creates a new String object

= vs == with Objects



- Assignment (=)
 - `a = b;` → both variables refer to the **same** object
 - No copy is made → **changes** through one **affect** the other

```
Car a = new Car();  
Car b = a;      // a and b point to the same object
```

- Define and use **copy constructor** to copy an object's content → (week 3)
- Testing for equality (==)
 - `a == b` → checks if two references point to the same object
 - Does not compare object content
 - Define and Use **equals()** method for content comparison → (week 3)

= vs == with Objects



```
Car a = new Car();  
Car b = new Car();
```

What is the output using:

- `a == b` → **false**
- `a.equals(b)` → **true** if `equals()` is properly overridden

The Constant null



- **null** is a special constant indicating a variable refers to **no** object.
- Used to **initialize** class-type variables when no object is yet assigned:

```
YourClass yourObject = null;
```
- Default value for uninitialized class-type variables.
- null is **not** an object. It is a placeholder with **no memory reference**
- To test if a variable is null, use `==` or `!=` (instead of `equals()`).
 - We are testing whether or not the variable refers to something, not testing its value.

```
if (myObject == null) { ... }
```
- ⚠ Accessing instance variables or non-static methods on a null reference causes a `NullPointerException`.

The new Operator and Anonymous Objects



- The **new** operator:
 - Invokes a constructor to create and initialize an object.
 - Returns a reference to the object's memory location.
- **Anonymous** Object is an object whose reference is **not** assigned to a variable
 - Useful for one-time method arguments:
 - Example:

```
ToyClass variable1 = new ToyClass("Joe", 42);  
if (variable1.equals(new ToyClass("JOE", 42))) { ... }
```
 - Ideal for short-lived, temporary use.



THE UNIVERSITY OF
MELBOURNE

Live Coding – Part 2 (call-by-reference)

Live Coding Instruction



Reuse the MyClass defined before.

In the main(), create one instance `class_1` and a reference variable `class_2` pointing to `class_1`.

Change the value of any variable in `class_1`, then print the corresponding variable's value in `class_2`.

- Print both `class_1` and `class_2` variables, and check if they are equal using `==`.
- Create a new instance `class_3`, check if `class_1` equals `class_3` using `==`.

Why?

Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 5)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 6 and 7)
- Classes and Objects (accessible on 14-02-2024) Oracle's Java Documentation. Available at: <https://docs.oracle.com/javase/tutorial/java/javaOO/index.html>

(c) The University of Melbourne 2023

This material is protected by copyright. Unless indicated otherwise, the University of Melbourne owns the copyright subsisting in the work. Other than for the purposes permitted under the Copyright Act 1968, you are prohibited from downloading, republishing, retransmitting, reproducing or otherwise using any of the materials included in the work as standalone files. Sharing University teaching materials with third-parties, including uploading lecture notes, slides or recordings to websites constitutes academic misconduct and may be subject to penalties. For more information: [Academic Integrity at the University of Melbourne](#)

Requests and enquiries concerning reproduction and rights should be addressed to the University Copyright Office, The University of Melbourne: copyright-office@unimelb.edu.au

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!



THE UNIVERSITY OF
MELBOURNE